Fully Interpretable Minimal Transformers: From Geometry to Algorithm

Toviah Moldwin, Raneem Mahajne, and Idan Segev

Edmond and Lily Safra Center for Brain Sciences, The Hebrew University of Jerusalem

Code & training movies on GitHub

Abstract

We present a framework for building and interpreting minimal transformer models. By constraining a transformer's embedding dimension and head size to 2, we enable full two-dimensional visualization of its internal representations. Embeddings, query/key/value transforms, attention outputs, residual streams, and decision boundaries can all be seen directly. Our central claim is that the learned geometry implies an algorithm; the arrangement of points and boundaries in $\mathbb{R}^2$ can be read as a step-by-step procedure. We train a transformer on a simple task where it must produce the most recently observed even number whenever the '+' operator appears in a sequence of digits. Once trained, we visually walk through every step of the transformer's computation. We show how the model embeds the tokens and their respective positions in the sequence, transforms them via the Q, K, and V matrices, uses the dot product between the Q and K representations to form the attention matrix, and uses the attention matrix to select values that move the representation of each input token to the region of the domain of the output layer that will correctly predict the next token. We introduce a suite of interpretability visualizations that make the algorithmic interpretation of this procedure explicit. Our framework offers a pedagogical and experimental testbed to explore how transformers use informational geometry to implement next-token prediction.

Model Architecture

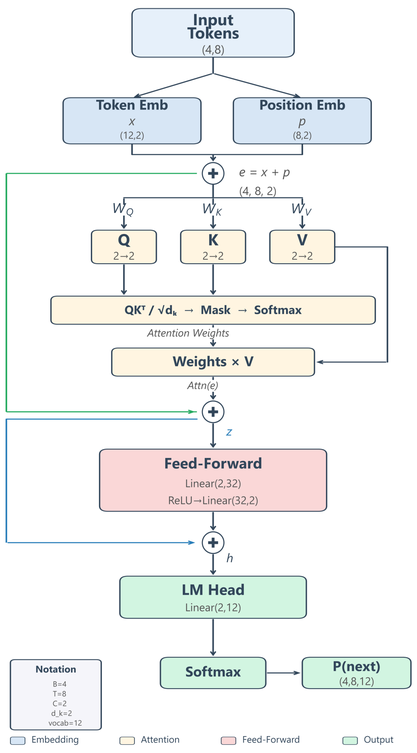

Figure 1. Architecture of the minimal transformer. Every component operates entirely in $\mathbb{R}^2$.

Training Data

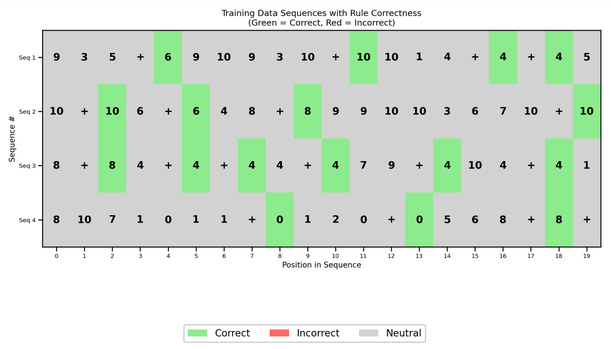

Figure 2. Sample training sequences as heatmaps. Green: constrained position (immediately after +) with the correct next token (the most recent even number). Gray: unconstrained position (any token is valid).

Learning Curve

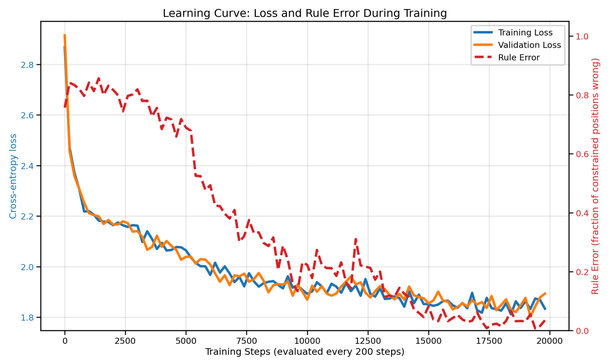

Figure 3. Training dynamics over 20,000 steps. Left axis: cross-entropy loss. Right axis: rule error — the fraction of constrained positions (tokens immediately following +) where the model's top predicted token is not the target (the most recent even number). Rule error near 90% is chance level; near 0% indicates the rule is learned.

Generated Sequences

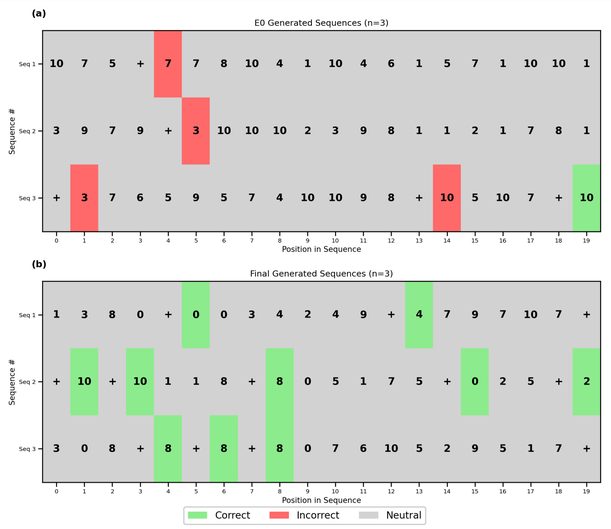

Figure 4. Model-generated sequences at initialization (top) vs. after training (bottom). Green = correct at constrained positions (after +), red = wrong at constrained positions, gray = unconstrained. Top: at step 0, constrained positions are mostly red (random predictions). Bottom: after training, constrained positions are mostly green (correct "last even" outputs).

Learned Embeddings

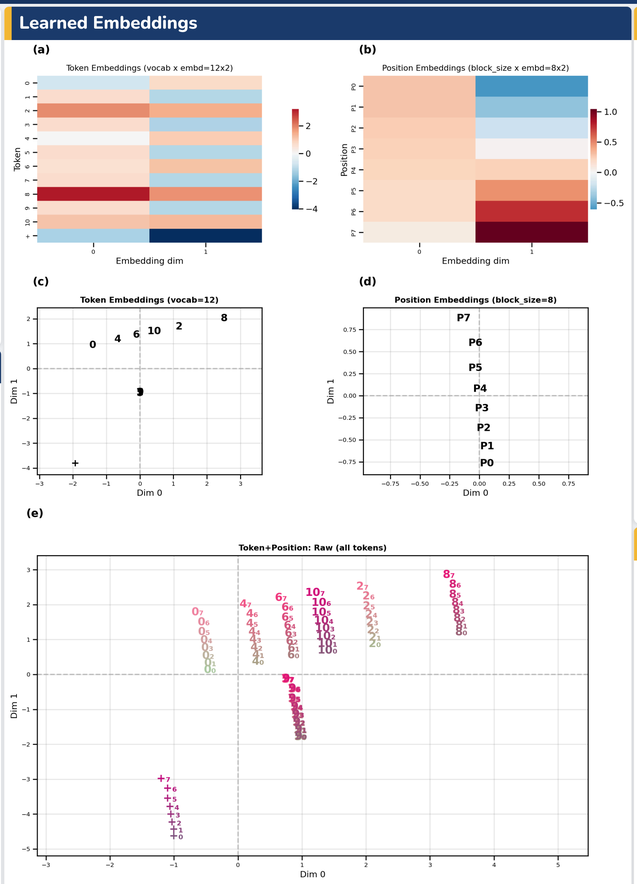

Figure 5. Learned embeddings. (a) Token embeddings $\mathbf{x}_i$ shown as heatmap. (b) Position embeddings $\mathbf{p}_i$ shown as heatmap. (c) Combined token+position embeddings shown as scatterplot. (d) Positions $\mathbf{p}_i$ shown as scatterplot. (e) Token+position embeddings $\mathbf{e}_i$ shown as scatterplot. Positions are indicated as subscripts next to each token e.g. $+_5$ is the + token at position 5.

Q, K, V Projections

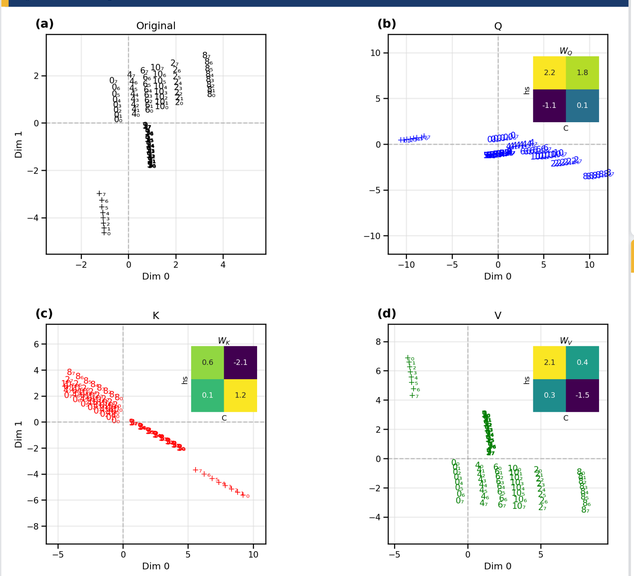

Figure 6. QKV projections. (a) Original combined token+position embeddings (as in Figure 5e). (b) Query-space (blue), with W_Q inset. (c) Key-space (red), with W_K inset. (d) Value-space (green), with W_V inset. All 96 token+position points in each panel.

Who Attends to Whom: Query–Key Geometry

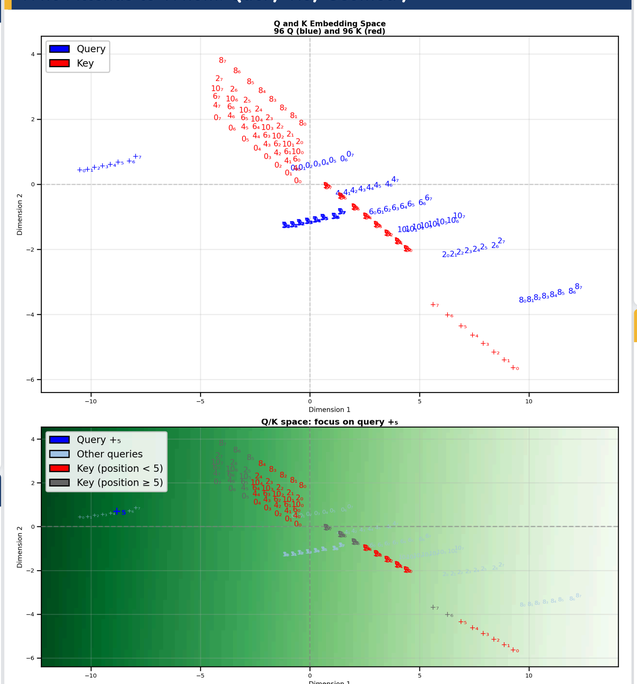

Figure 7. Query–key geometry in Q/K space. **Top:** joint query–key space (blue: queries; red: keys; labels show token and position). **Bottom:** Focus on the query $+_5$ (blue); background color indicates the dot product of the $+_5$ query with every point in space. Keys with position ≥ 5 are grayed due to the causal mask.

QK Scores for + Queries

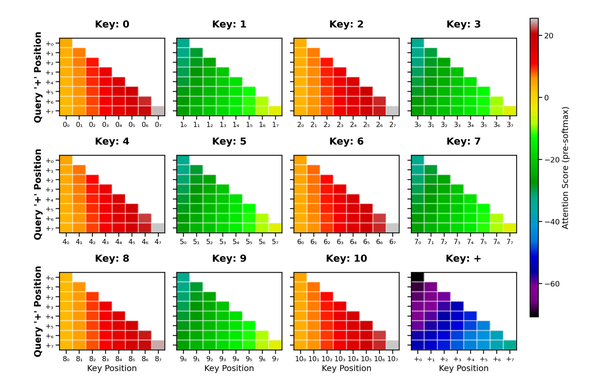

Figure 8. Pre-softmax query–key scores $QK^\top$, restricted to + queries. Each subplot shows an 8×8 matrix of dot products: + query positions (rows) vs. key positions (columns) for a particular key token.

Output Probability Landscape

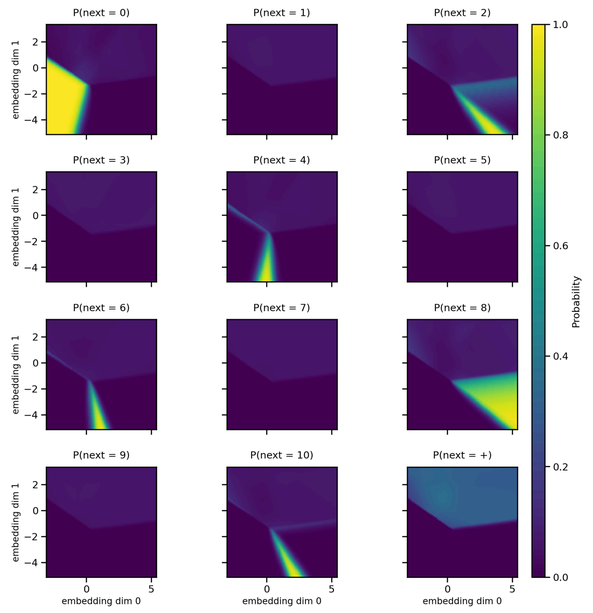

Figure 9. Output probability landscape (per-unit view). Each subplot shows the softmax probability $P(\text{next} = \text{token})$ over the 2D plane for one output token (digits 0–10 and the + operator).

Output Landscape Summary

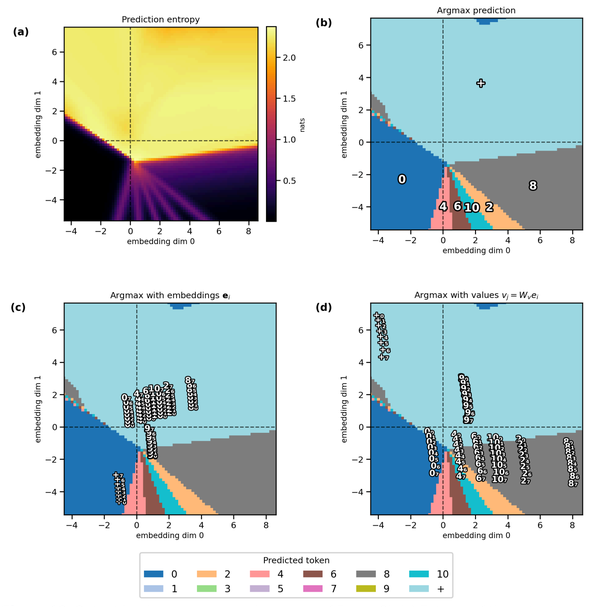

Figure 10. Output landscape summary (summary of Figure 9). (a) Entropy of the output distribution (nats); high in the indifference region (top half), near zero in even-number target zones (bottom half). (b) Argmax prediction map: color and annotation indicate which token has the highest predicted probability at each point. (c) Argmax map with combined embeddings $\mathbf{e}_i$ overlaid. (d) Argmax map with value-transformed vectors $\mathbf{v}_j$ overlaid.

Tracing a Sequence 4 1 + 4 6 9 5 +: Embeddings

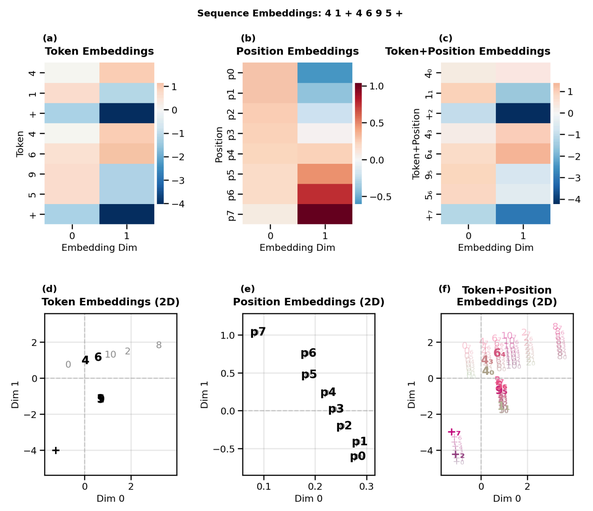

Figure 11. Embeddings for the demo sequence 4 1 + 4 6 9 5 +. **a–c:** Embedding heatmaps for tokens, positions and token+position. **d–f:** 2D scatter plot views of tokens, positions, and token+position combinations, shown over a backdrop of all possible tokens, positions, and token+position embeddings (light gray).

Tracing: Q & K

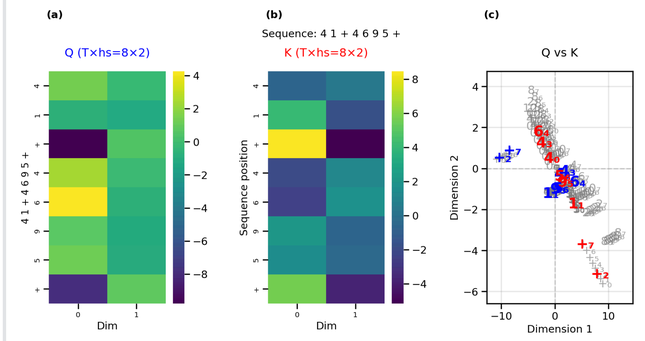

Figure 12. Q and K for the demo sequence. (a) Q heatmap, (b) K heatmap, (c) scatterplot of queries (blue), and keys (red) for tokens within our test sequence against a backdrop of all 96 possible queries and keys (gray).

Tracing: Attention Computation

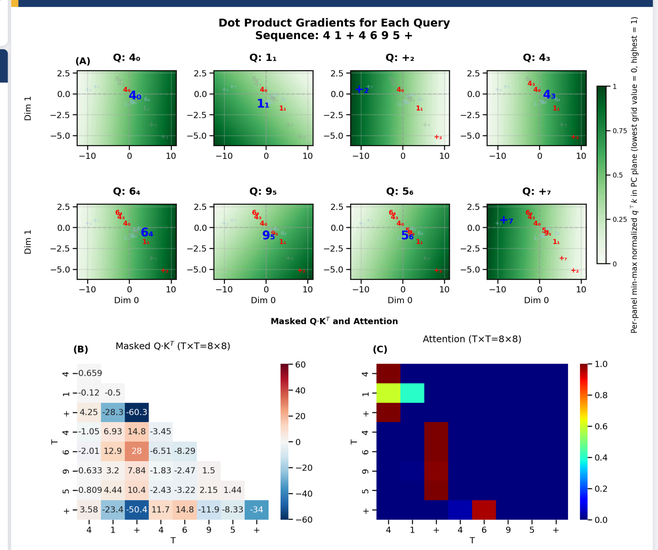

Figure 13. Attention computation for the demo sequence. (a) The background of each panel displays the dot product of a specific query (blue) in the test sequence with every point in space (green = high, white = low). The unmasked (red) and masked (gray) keys are overlaid on this space to show their dot products with the selected query. (b) The masked $Q \cdot K^\top$ matrix, (c) the attention matrix after normalization by $\sqrt{d_k}$ and applying softmax.

Tracing: Values & Attention Output

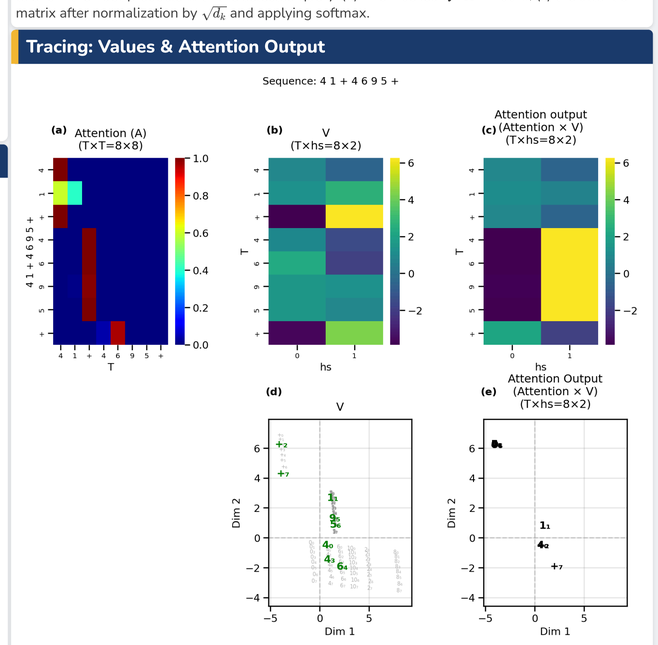

Figure 14. Values and attention output for the demo sequence. (a) Attention weight matrix as in Figure 13c, (b) Values for each digit shown as heatmaps. (c) Attention outputs shown as heatmaps. (d) Values shown as scatter plot. (e) Attention output shown as scatter plot.

Tracing: Next-Token Prediction

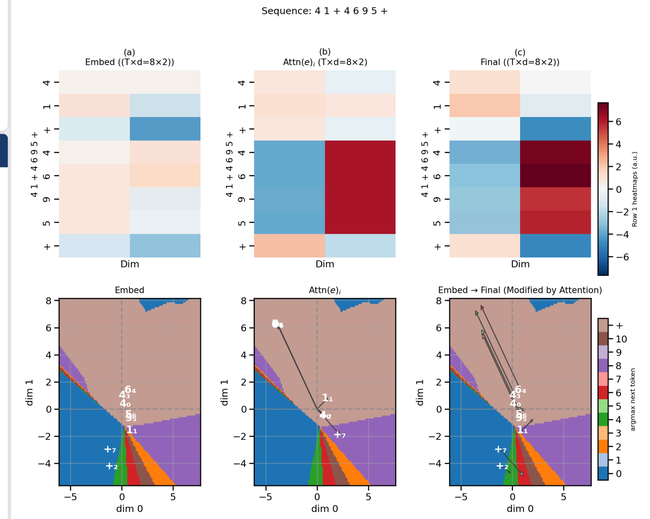

Figure 15. Predicting the next token by summing the input embeddings and attention vectors. (a) Top: Heatmaps illustrating the 2D values for the input embeddings (**e**) for each token in the test sequence. Bottom: Input embeddings in the 2D output space, where the background colors represent the most likely next-token prediction. (b) Top: The attention vectors ($\text{Attn}(\mathbf{e})$). Bottom: The attention weights shown as arrows from the origin. (c) Top: The sum of the input embeddings and the attention vectors (**z**) as heatmap. Bottom: Input embeddings (white annotations) are moved by the attention vectors (arrows) to the correct region for predicting the next token.

Contact: Raneem Mahajne — Edmond and Lily Safra Center for Brain Sciences, The Hebrew University of Jerusalem

Key references: Vaswani et al. (2017); Radford et al. (2018); Elhage et al. (2021); Nanda et al. (2023); Weiss et al. (2021). Full list in paper.